

AcquirerX Deep-Dive 7/10 — Risk Service

Service: `risk-service` **Port:** `9094` **Database:** `acquirerx_risk` **Role:** Real-time fraud and risk decisioning — block, review, or allow each transaction

1. What It Is

Risk-service is the **decision engine** that sits between the transaction switch and approval. Every authorize request triggers a synchronous Feign call to risk-service, which returns one of three verdicts:

- **ALLOW** — transaction is safe, proceed
- **REVIEW** — approve but flag for human review
- **BLOCK** — refuse the transaction

Risk-service owns three things:

1. **RiskRule** — configurable rules that trigger on certain conditions (amount limits, etc.)
 2. **Blacklist** — banned PANs, terminals, merchants
 3. **RiskEvent** — log of every triggered rule for audit and analysis
-

2. Why It's a Separate Service

- **Risk policy changes frequently** — risk team updates rules without affecting other services
 - **Independent latency profile** — risk decisioning needs to be fast (< 100ms), separate scaling
 - **Domain isolation** — fraud logic is its own world, kept away from CRUD-heavy services
 - **Auditability** — every risk decision logged in one place
 - **Future evolution** — risk engines often grow into ML models, third-party integrations (Sift, Forter); isolating them now makes future swaps easy
-

3. Tech Stack

- Spring Boot 3.x

- Spring Data JPA + MySQL
 - Lombok
 - No Feign clients here — risk-service is a leaf service (called by transaction-service, doesn't call others)
-

4. Code Structure

```
risk-service/src/main/java/com/acquirerx/risk/  
├─ RiskServiceApplication.java  
└─ risk/  
    ├─ controller/RiskController.java  
    ├─ dto/ (RiskCheck, RiskEvent, RiskRule, Blacklist, RiskSummary DTOs)  
    ├─ entity/  
    │   ├─ RiskEvent.java  
    │   ├─ RiskRule.java  
    │   └─ Blacklist.java  
    └─ service/RiskService.java
```

A small, focused service. One controller, one service class, three entities.

5. Entities

5.1 `RiskRule` (table: `risk_rule`)

A configurable rule that triggers under certain conditions.

Field	Type	Purpose
<code>riskRuleId</code>	Long PK	
<code>name</code>	String	e.g., "High-value transaction limit"
<code>expression</code>	String	Stored but not evaluated — see below
<code>maxAmount</code>	Double	The actual rule trigger — if amount exceeds this, rule fires
<code>severity</code>	String	CRITICAL / HIGH / MEDIUM / LOW
<code>action</code>	String	ALLOW / REVIEW / BLOCK
<code>active</code>	Boolean	Easy enable/disable

Important honest note about the `expression` field: The schema includes an `expression` field — the original intent was to support dynamic expressions (something like SpEL or a mini-DSL: `"amount > 50000 AND mcc = '7995'"`). In the current implementation, the expression is **stored but never parsed or evaluated**. The actual rule trigger is purely based on `maxAmount`.

This is documented as a known design limit. Future enhancement could plug in a real expression engine (Spring SpEL, Janino, MVEL, or Drools).

Why severity matters: Even though the action determines what happens, severity is recorded for reporting. "We had 50 CRITICAL rule triggers today" tells the risk team they have a problem.

5.2 `Blacklist` (table: `blacklist`)

A list of banned values across three categories.

Field	Type	Purpose
<code>blacklistId</code>	Long PK	
<code>type</code>	String	PAN / TERMINAL / MERCHANT
<code>value</code>	String	The banned identifier
<code>reason</code>	String	Why it was blacklisted
<code>active</code>	Boolean	Soft delete flag
<code>createdAt</code>	LocalDateTime	

Why three blacklist types:

Type	What it bans	Real-world example
PAN	A specific masked card number	Stolen card reported by issuer
TERMINAL	A specific TID	Compromised POS device
MERCHANT	A specific merchant ID	Chronic chargeback abuser

A single transaction can match any/all three. Risk-service checks all three on every transaction.

Why PAN values are stored masked: When adding a PAN to the blacklist,

`MaskingUtil.maskPan(value)` is applied first. This way, blacklist values match the masked format used in transaction PANs. The actual full PAN never enters the database.

5.3 `RiskEvent` (table: `risk_event`)

Audit log of every rule that triggered on a transaction.

Field	Type	Purpose
<code>riskEventId</code>	Long PK	
<code>txnId</code>	Long FK	Reference to the AuthMessage in transaction-service
<code>txnAmount</code>	Double	Amount at time of event
<code>rule</code>	RiskRule (FK)	Which rule fired
<code>score</code>	Integer	Calculated risk score (0-100)
<code>result</code>	String	The action that was taken (ALLOW/REVIEW/BLOCK)
<code>eventDate</code>	LocalDateTime	When

Why store the score: The score is a calculated metric (`min((amount/maxAmount) * 50, 100)`). Storing it lets you build dashboards: "average risk score this week," "spike in scores after midnight."

Why a JPA `@ManyToOne RiskRule` here (not just `Long ruleId`): This is the only place in AcquirerX where a JPA relationship crosses entity types. Why? Because RiskEvent and RiskRule are both in the same service, same database. Cross-service JPA joins are forbidden, but within

the same service, JPA relationships are fine and convenient (the response DTO can include both rule ID and rule name without an extra fetch).

6. The RiskService — checkRisk Flow

You provided the full code. Step-by-step walkthrough:

6.1 Entry point — `checkRisk(amount, panMasked, tid)`

This is the method called via Feign from transaction-service during every authorize request.

```
java

public RiskCheckResultDTO checkRisk(Double amount, String panMasked, String tid) {

    // — Step 1: Terminal blacklist check —
    if (tid != null) {
        Optional<Blacklist> terminalBlacklist =
            blacklistRepository.findByTypeAndValueAndActiveTrue("TERMINAL", tid)
        if (terminalBlacklist.isPresent()) {
            log.warn("Blacklisted terminal: {}", tid);
            return new RiskCheckResultDTO("BLOCK", 100, "Terminal is blacklisted");
        }
    }
}
```

Why terminal check first:

- Cheap query (indexed lookup on (type, value, active))
- Highest-priority block — a compromised terminal is a major incident
- Returns immediately on hit; no need to evaluate further

```
java
```

```
// — Step 2: PAN blacklist check —
if (panMasked != null) {
    String normalizedPan = MaskingUtil.maskPan(panMasked);
    Optional<Blacklist> panBlacklist =
        blacklistRepository.findByTypeAndValueAndActiveTrue("PAN", normalizedPan);
    if (panBlacklist.isPresent()) {
        log.warn("Blacklisted PAN attempted");
        return new RiskCheckResultDTO("BLOCK", 100, "Card is blacklisted");
    }
}
```

Why normalize via maskPan: Defensive — the caller passes whatever PAN form they have (could be raw, could be partially masked, could be already masked). `maskPan` is idempotent (calling it on an already-masked value returns the same masked value). Normalizing ensures the comparison hits the same string format that was used when adding to the blacklist.

java

```
// — Step 3: Rule evaluation —
List<RiskRule> activeRules = riskRuleRepository.findByActiveTrue();

String worstResult = "ALLOW";
int highestScore = 0;
String reason = "All checks passed";

for (RiskRule rule : activeRules) {
    if (amount > rule.getMaxAmount()) {
        int score = calculateScore(amount, rule.getMaxAmount());

        if (isMoreSevere(rule.getAction(), worstResult)) {
            worstResult = rule.getAction();
            highestScore = score;
            reason = "Rule triggered: " + rule.getName()
                + " (amount " + amount
                + " exceeds limit " + rule.getMaxAmount() + ")";
        }
    }
}
```

The actual rule logic — line-by-line:

- Fetch all active rules

- Initialize: worst result = ALLOW, score = 0, reason = "all passed"
- For each rule: if `amount > maxAmount`, the rule has triggered
- Calculate the score based on how much the amount exceeds the limit
- If this rule's action is *more severe* than the current worst, escalate

Why "more severe": Multiple rules might trigger. If Rule A says REVIEW and Rule B says BLOCK, the BLOCK wins. The transaction is blocked.

Severity ranking:

```
java

private int severityScore(String action) {
    return switch (action.toUpperCase()) {
        case "BLOCK" -> 3;
        case "REVIEW" -> 2;
        case "ALLOW" -> 1;
        default -> 0;
    };
}

private boolean isMoreSevere(String a, String b) {
    return severityScore(a) > severityScore(b);
}
```

BLOCK (3) > REVIEW (2) > ALLOW (1). The most severe matched rule wins.

```
java

log.info("Risk check: amount={}, result={}, score={}", amount, worstResult, high
return new RiskCheckResultDTO(worstResult, highestScore, reason);
}
```

Final log + return.

6.2 The score calculation

```
java
```

```
private int calculateScore(double amount, double maxAmount) {  
    double ratio = amount / maxAmount;  
    return (int) Math.min(ratio * 50, 100);  
}
```

How it works:

- $\text{ratio} = \text{amount} / \text{maxAmount}$ — how many times over the limit
- Multiply by 50 — at exactly the limit, score is 50; double the limit, score is 100
- Cap at 100

Examples:

Amount	maxAmount	Ratio	Score
1000	50000	0.02	1
50000	50000	1.0	50
75000	50000	1.5	75
100000	50000	2.0	100
500000	50000	10.0	100 (capped)

Why this formula:

- Linear above the limit, so a slightly-over transaction scores low and a wildly-over one scores high
- Cap at 100 so the score is bounded
- Easy for the risk team to understand ("score of 75 = 50% over limit")

7. evaluateAndSave — Persisting Risk Events

```
java
```



```

public void evaluateAndSave(Long txnId, Double amount) {
    List<RiskRule> activeRules = riskRuleRepository.findByActiveTrue();

    for (RiskRule rule : activeRules) {
        if (amount > rule.getMaxAmount()) {
            RiskEvent event = new RiskEvent();
            event.setTxnId(txnId);
            event.setRule(rule);
            event.setScore(calculateScore(amount, rule.getMaxAmount()));
            event.setResult(rule.getAction());
            riskEventRepository.save(event);
            log.info("Risk event saved: txnId={}, rule={}, result={}",
                    txnId, rule.getName(), rule.getAction());
        }
    }
}

```

Why this is a separate method: `checkRisk` is the synchronous decision call — it must be fast, returns the verdict to the switch. Saving every triggered rule as a `RiskEvent` is a separate concern and could be done after the fact.

When this is called: After the transaction-service has saved the `AuthMessage`, it can call this with the resulting `txnId` (= `authId`). Risk-service then writes one `RiskEvent` per triggered rule.

Why "for each triggered rule, save event" (not just one event): A high-value transaction might trigger multiple rules ("over ₹50,000," "off-hours," "high-risk MCC"). Each is its own event for granular reporting.

Note: Currently this method is defined but not actively called from transaction-service in the auth flow. If you want full event tracking, transaction-service would need to call `evaluateAndSave` after `checkRisk`, or the call would happen via an event-driven pattern.

8. Rule Management

8.1 createRule

```

java

```

```

public RiskRuleResponseDTO createRule(RiskRuleRequestDTO dto) {
    RiskRule rule = new RiskRule();
    rule.setName(dto.getName());
    rule.setExpression(dto.getExpression());
    rule.setMaxAmount(dto.getMaxAmount());
    rule.setSeverity(dto.getSeverity());
    rule.setAction(dto.getAction());
    rule.setActive(true);

    return toRuleResponse(riskRuleRepository.save(rule));
}

```

Standard create. Note that `active` defaults to true.

8.2 deactivateRule

```

java

public RiskRuleResponseDTO deactivateRule(Long ruleId) {
    RiskRule rule = riskRuleRepository.findById(ruleId)
        .orElseThrow(() -> new ResourceNotFoundException("Rule not found"));
    rule.setActive(false);
    return toRuleResponse(riskRuleRepository.save(rule));
}

```

Soft delete via the `active` flag. Rules are never hard-deleted because:

- Past RiskEvents reference them (FK relationship)
- Auditors might ask "what was the rule on Sep 5?"

9. Blacklist Management

9.1 addToBlacklist

```

java

```

```

public BlacklistResponseDTO addToBlacklist(BlacklistRequestDTO dto) {
    String normalizedValue = "PAN".equalsIgnoreCase(dto.getType())
        ? MaskingUtil.maskPan(dto.getValue())
        : dto.getValue();

    blacklistRepository.findByTypeAndValueAndActiveTrue(dto.getType(), normalizedValue)
        .ifPresent(b -> {
            throw new IllegalStateException(
                dto.getType() + " already blacklisted");
        });

    Blacklist entry = new Blacklist();
    entry.setType(dto.getType());
    entry.setValue(normalizedValue);
    entry.setReason(dto.getReason());

    return toBlacklistResponse(blacklistRepository.save(entry));
}

```

Three things happening:

1. **PAN normalization** — for type=PAN, mask the value first. For TERMINAL/MERCHANT, store as-is.
2. **Duplicate check** — prevent multiple active entries for same value
3. **Save** — `active=true` default, `createdAt` auto-set

Why duplicate check: Without it, an admin could add the same PAN multiple times (different reasons), polluting the blacklist. The `findByTypeAndValueAndActiveTrue` query returns the matching entry if one exists.

9.2 removeFromBlacklist

```

java

public BlacklistResponseDTO removeFromBlacklist(Long blacklistId) {
    Blacklist entry = blacklistRepository.findById(blacklistId)
        .orElseThrow(() -> new ResourceNotFoundException(
            "Blacklist entry not found"));
    entry.setActive(false);
    return toBlacklistResponse(blacklistRepository.save(entry));
}

```

Soft delete via `active=false`. The record stays for audit. The `checkRisk` method only matches `active=true`, so deactivated entries no longer block.

10. RiskSummary — The Dashboard Query

java

```
public RiskSummaryDTO getRiskSummary() {
    LocalDateTime todayStart = LocalDate.now().atStartOfDay();

    long totalEvents = riskEventRepository.count();
    long totalBlocked = riskEventRepository.countByResult("BLOCK");
    long totalReview = riskEventRepository.countByResult("REVIEW");
    long totalAllow = riskEventRepository.countByResult("ALLOW");

    long todayEvents = riskEventRepository.countByDateAfter(todayStart);
    long todayBlocked = riskEventRepository.countByResultAndDateAfter("BLOCK", todayStart);
    long todayReview = riskEventRepository.countByResultAndDateAfter("REVIEW", todayStart);

    long panCount = blacklistRepository.countByTypeAndActiveTrue("PAN");
    long terminalCount = blacklistRepository.countByTypeAndActiveTrue("TERMINAL");
    long merchantCount = blacklistRepository.countByTypeAndActiveTrue("MERCHANT");

    long blockRules = riskRuleRepository.countByActionAndActiveTrue("BLOCK");
    long reviewRules = riskRuleRepository.countByActionAndActiveTrue("REVIEW");
    long activeRules = blockRules + reviewRules +
        riskRuleRepository.countByActionAndActiveTrue("ALLOW");

    double blockRate = totalEvents > 0
        ? Math.round((totalBlocked * 100.0 / totalEvents) * 100.0) / 100.0
        : 0.0;

    return new RiskSummaryDTO(
        totalEvents, totalBlocked, totalReview, totalAllow,
        todayEvents, todayBlocked, todayReview,
        panCount, terminalCount, merchantCount,
        activeRules, blockRules, reviewRules,
        blockRate
    );
}
```

Why this exists: Drives the Risk dashboard in the frontend. Shows:

- Lifetime + today's risk activity
- Blacklist counts per type
- Active rule counts per action
- Block rate (e.g., "0.85% of transactions blocked")

Why all in one method: A dashboard needs all these metrics together. Doing one query per metric would be 13 round trips. This batches them but still hits the DB many times. In a high-load production system, you'd cache this with a 30-second TTL.

Block rate rounding: `Math.round(x * 100) / 100` is the standard Java idiom for rounding to 2 decimal places.

11. Repositories

11.1 RiskRuleRepository

```
java

public interface RiskRuleRepository extends JpaRepository<RiskRule, Long> {
    List<RiskRule> findByActiveTrue();
    Page<RiskRule> findByActiveTrue(Pageable pageable);
    long countByActionAndActiveTrue(String action);
}
```

11.2 BlacklistRepository

```
java

public interface BlacklistRepository extends JpaRepository<Blacklist, Long> {
    Optional<Blacklist> findByTypeAndValueAndActiveTrue(String type, String value);
    Page<Blacklist> findByActiveTrue(Pageable pageable);
    Page<Blacklist> findByTypeAndActiveTrue(String type, Pageable pageable);
    long countByTypeAndActiveTrue(String type);
}
```

11.3 RiskEventRepository

```
java
```

```

public interface RiskEventRepository extends JpaRepository<RiskEvent, Long> {
    long countByResult(String result);

    @Query("SELECT COUNT(e) FROM RiskEvent e WHERE e.eventDate >= :start")
    long countByDateAfter(@Param("start") LocalDateTime start);

    @Query("SELECT COUNT(e) FROM RiskEvent e WHERE e.result = :result AND e.eventDate >= :start")
    long countByResultAndDateAfter(@Param("result") String result,
                                    @Param("start") LocalDateTime start);

    @Query("SELECT e FROM RiskEvent e WHERE " +
        "(:result IS NULL OR e.result = :result) AND " +
        "(:minScore IS NULL OR e.score >= :minScore) AND " +
        "(:maxScore IS NULL OR e.score <= :maxScore) AND " +
        "(:from IS NULL OR e.eventDate >= :from) AND " +
        "(:to IS NULL OR e.eventDate <= :to) AND " +
        "(:txnId IS NULL OR e.txnId = :txnId) AND " +
        "(:ruleId IS NULL OR e.rule.riskRuleId = :ruleId)")
    Page<RiskEvent> findByFiltersPaged(...);
}

```

The filter query uses the optional-filter pattern (`(:param IS NULL OR field = :param)`) consistent with other services.

12. The RiskController

12.1 Endpoints

— Rules —

GET	/risk/rules	(paged)
POST	/risk/rules	(create)
PATCH	/risk/rules/{id}/deactivate	

— Events —

GET	/risk/events	(paged, all events)
POST	/risk/events/search	(paged with filter)

— Blacklist —

GET	/risk/blacklist?type=...	(paged, optional type filter)
POST	/risk/blacklist	(add)

```
DELETE /risk/blacklist/{id} (soft delete)
```

— Manual Check —

```
POST /risk/check?amount=&panMasked=&tid=
```

— Summary —

```
GET /risk/summary
```

12.2 The /risk/check endpoint

```
java

@PostMapping("/check")
public ApiResponse<RiskCheckResultDTO> check(
    @RequestParam Double amount,
    @RequestParam String panMasked,
    @RequestParam String tid) {
    return ApiResponse.success("Risk checked",
        riskService.checkRisk(amount, panMasked, tid));
}
```

This is the endpoint hit via Feign by transaction-service on every authorize. Query params are used (not request body) because:

- Simple: just a few primitive values
- Stateless: easy to log and trace
- Cacheable: in theory, repeated identical params could be cached (though risk decisions usually shouldn't be)

13. application.properties

```
properties
```

```
server.port=9094
spring.application.name=risk-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

spring.datasource.url=jdbc:mysql://localhost:3306/acquirerx_risk?createDatabaseIfNot
spring.datasource.username=root
spring.datasource.password=kail3114
spring.jpa.hibernate.ddl-auto=update

server.servlet.context-path=/api/v1
```

No Feign config needed — risk-service doesn't call any other service.

14. The Full Risk Decision Flow

1. Transaction-service calls risk-service:
POST /api/v1/risk/check?amount=75000&panMasked=699006X*X*4178&tid=00012345
2. Risk-service:
 - Step 1: TERMINAL blacklist check → blacklist for "00012345"? No.
 - Step 2: PAN blacklist check
 - normalizedPan = MaskingUtil.maskPan("699006X*X*4178") = "699006X*X*4178"
 - (idempotent)
 - blacklist for "699006X*X*4178"? No.
 - Step 3: Active rules
 - Rule A: name="Limit", maxAmount=50000, action="REVIEW"
 - 75000 > 50000 → triggered
 - score = min((75000/50000)*50, 100) = 75
 - REVIEW > ALLOW (current worst) → escalate
 - Rule B: name="Hard cap", maxAmount=100000, action="BLOCK"
 - 75000 > 100000? No. Skip.
 - Rule C: ...
 - Final: result=REVIEW, score=75, reason="Rule triggered: Limit..."
 - Return RiskCheckResultDTO
3. Transaction-service receives:
{ result: "REVIEW", score: 75, reason: "Rule triggered: Limit (...)" }
4. SwitchService logic:

- REVIEW means "approve but flag" – log warning, set status=APPROVED
- AuthMessage saved with riskScore=75, riskReason="Rule triggered: Limit..."

15. Why This Design Is Robust

- **Blacklist checked first** — fastest reject path, immediate BLOCK on hit
- **Severity ranking** — multiple rules can fire, the most severe wins
- **PAN normalization** — never store raw PANs, even in blacklists
- **Active flag everywhere** — soft delete preserves history
- **Score calculation is deterministic** — same input always yields same score, easy to test
- **Graceful degradation in caller** — if risk-service is down, transaction-service defaults to ALLOW (configurable fail-open posture)
- **Self-contained** — no Feign calls out, no dependencies, simplest deployment

16. Honest Design Limits (and Future Enhancements)

Limit	Current behavior	Possible enhancement
<code>expression</code> field unused	Only <code>maxAmount</code> matters	Plug in SpEL/Drools/Janino expression engine
Rules only check amount	Other dimensions (MCC, region, time-of-day) not used	Add filters to RiskRule schema
Sequential rule eval	All rules checked even after BLOCK	Short-circuit on BLOCK for performance
No velocity rules	Can't catch "5 swipes on same card in 1 minute"	Add Redis-based sliding-window counters
No ML scoring	Pure rules	Integrate model serving (TensorFlow Serving, SageMaker)
No third-party integration	All in-house	Plug in Sift, Forter, Kount

These are real-world risk engine features. AcquirerX's current design is intentionally simple but provides hooks (the `expression` field, the score concept) for future evolution.

17. Common Pitfalls

Pitfall	Symptom	Fix
Adding raw PAN to blacklist	Won't match transactions (which use masked PANs)	Always mask before insert (already done)
Forgetting to set <code>active=true</code> on rule create	Rule never fires	Default in service code
Multiple BLOCK rules overlapping	Same outcome but multiple events written	Acceptable — events are valuable for analysis
Score calculation underflow on tiny amounts	Score = 0 even when over limit	Acceptable — small overage is low risk
Risk-service down in production	Transactions auto-allowed (fail-open)	Add monitoring + alerts

18. Summary

Aspect	Detail
Purpose	Real-time risk decisioning + blacklist + rules
Port	9094
DB	<code>acquirerx_risk</code> (3 tables: rule, event, blacklist)
Tech	Spring Boot, JPA, MySQL
Outputs	ALLOW / REVIEW / BLOCK with score 0-100
Severity ranking	BLOCK > REVIEW > ALLOW
Score formula	<code>min((amount/maxAmount) * 50, 100)</code>
Blacklist types	PAN, TERMINAL, MERCHANT
Self-contained	No Feign calls out

Aspect	Detail
Called by	transaction-service (every txn)
Honest limit	expression field stored but not evaluated

Next: Settlement Service — batches, payouts, the EOD scheduler.